PROCÉDURE · MON INFRASTRUCTURE HAUTE DISPONIBILITÉ · RÉSEAU ET SÉCURITÉ

MO-5 — Clés SSH, durcissement et moindre privilège

Connexion SSH par clé, durcissement des serveurs, comptes à privilèges minimaux et gestion des secrets.

> 🗂️ Document de mon dossier E6 (BTS SIO SISR), tel que je l'ai rédigé pendant la mise en place de mon laboratoire. Version du 10/10/2026. Les adresses sont celles de mon laboratoire : adapte-les au tien.

Connexion par clé, durcissement des serveurs, comptes à privilèges minimaux, gestion des secrets.

> 🔐 Aucun mot de passe ni clé dans ce document : les secrets sont remplacés par des marque-places.
>
> ⚠️ **Écarts avec le labo réel** (configuration relevée le 01/10/2026, voir le DAT §7.2) : HAProxy contrôle ses serveurs **toutes les 2 s** (`inter 2s fall 3 rise 2`), le contrôle SQL est un simple test de port (`tcp-check` , pas `mysql-check`), et il n'y a **ni page de statistiques (8404) ni en-tête `X-Served-By`** : les passages qui les citent décrivent une cible, pas l'existant. adm-01 est le contrôleur de domaine Windows : les commandes notées « sur adm-01 » se jouent depuis une session du bastion Guacamole.

Sécurité — SSH

Principes

Mesure	Raison
Authentification **par clé** uniquement	Pas de mot de passe à deviner ou à intercepter
Pas de connexion `root`	Traçabilité (qui a fait quoi) et `sudo` journalisé
SSH accepté **depuis vADMIN** uniquement	Règles pfSense + `AllowUsers` /pare-feu local
Clé **ed25519** protégée par une phrase de passe	Algorithme moderne ; clé volée inutilisable seule
Vérification des empreintes d'hôte	Protection contre l'usurpation de serveur

Clé publique ou clé privée ?

```
Poste d'administration                              Serveur (ex. bdd-01)
~/.ssh/id_ed25519      ← clé PRIVÉE : ne sort jamais du poste
~/.ssh/id_ed25519.pub  ── copiée vers ──►   ~/.ssh/authorized_keys
                                              (liste des clés PUBLIQUES autorisées)
```

- `authorized_keys` contient des **clés publiques** : il dit « j'accepte toute personne capable de prouver qu'elle détient la clé privée correspondante ».
- La clé **privée** ne doit **jamais** être copiée sur un serveur ni dans ce dépôt. La coller dans `authorized_keys` est une erreur : la connexion échoue (`Permission denied (publickey)`) et la clé privée est exposée. Voir INC-06.

Mise en place

SUR adm-01

```
ssh-keygen -t ed25519 -C "adm01-admin"          # phrase de passe recommandée
ssh-copy-id <ADMIN_USER>@10.0.20.1               # ajoute la clé PUBLIQUE dans authorized_keys
ssh <ADMIN_USER>@10.0.20.1                       # test
cp configs/ssh/linux-ssh-config.example ~/.ssh/config && chmod 600 ~/.ssh/config
ssh mysql01                                      # alias défini dans ~/.ssh/config
```

SUR CHAQUE VM — après avoir vérifié que la connexion par clé fonctionne :

```
sudo nano /etc/ssh/sshd_config.d/10-hardening.conf
```

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
X11Forwarding no
```

```
sudo sshd -t && sudo systemctl reload ssh
```

> Garder une session SSH ouverte pendant la modification : en cas d'erreur, elle permet de corriger sans passer par la console Proxmox.

Avertissement « REMOTE HOST IDENTIFICATION HAS CHANGED »

SSH mémorise l'empreinte de chaque serveur dans `~/.ssh/known_hosts`. Si elle change, SSH **refuse** la connexion : c'est exactement ce qui se produirait lors d'une attaque de l'homme du milieu.

Avant de supprimer l'ancienne empreinte avec `ssh-keygen -R`, **vérifier la cause** :

Cause légitime	Vérification
VM réinstallée ou recréée	Vous l'avez fait vous-même, récemment
IP réattribuée à une autre VM	Plan d'adressage, Proxmox
Clés d'hôte régénérées	Historique d'administration

Comparer avec l'empreinte réelle, lue **depuis la console Proxmox** de la VM (canal de confiance) :

```
# SUR LA VM (console Proxmox)
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Si elle correspond à celle annoncée dans l'avertissement, alors seulement :

```
# SUR adm-01
ssh-keygen -R 10.0.20.1
ssh mysql01      # accepter la nouvelle empreinte après comparaison
```

Détail : INC-07

Bastion

Le poste d'administration externe passe par **adm-01** (`ProxyJump`) : seul adm-01 doit être joignable depuis l'extérieur. Voir `configs/ssh/windows-ssh-config.example` (dépôt GitHub). Guacamole offre la même centralisation via un navigateur.

Sécurité — Durcissement

Mesures de durcissement par composant. Le statut distingue ce qui est **en place** (intégré à la configuration de référence v1.0.0 et aux fichiers de `configs/`) de ce qui est **recommandé** (non déployé) ou **futur**. Avant une soutenance, confirmer chaque ligne « En place » sur les VM réelles.

Toutes les VM Debian

Mesure	Commande / fichier	Statut
Mises à jour de sécurité	`sudo apt update && sudo apt upgrade`	En place (manuel)
Mises à jour automatiques de sécurité	`sudo apt install unattended-upgrades`	Recommandé
SSH par clé, sans root	ssh.md	En place
Services inutiles désactivés	`systemctl list-units --type=service --state=running`	À vérifier par VM
Horloge synchronisée	`timedatectl` (systemd-timesyncd)	En place
Journaux persistants	`journalctl --disk-usage`	Par défaut Debian 12

nftables sur les nœuds Galera

Exemple de pare-feu local (statut : **recommandé**, à tester avant déploiement). Il restreint les ports Galera aux deux autres nœuds, et 3306 à HAProxy.

SUR bdd-01 — `/etc/nftables.conf` (adapter `self` pour bdd-02 et bdd-03)

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    set galera_peers { type ipv4_addr; elements = { 10.0.20.1, 10.0.20.2, 10.0.20.3 } }

    chain input {
        type filter hook input priority 0; policy drop;
        iif lo accept
        ct state established,related accept
        ip protocol icmp accept
        ip saddr 10.0.30.0/24 tcp dport 22 accept                    # SSH depuis vADMIN
        ip saddr 10.0.10.2 tcp dport 3306 accept                     # SQL depuis HAProxy
        ip saddr @galera_peers tcp dport { 3306, 4444, 4567, 4568 } accept
        ip saddr @galera_peers udp dport 4567 accept
    }
    chain forward { type filter hook forward priority 0; policy drop; }
    chain output  { type filter hook output priority 0; policy accept; }
}
```

```
sudo nft -c -f /etc/nftables.conf        # vérification de syntaxe
sudo systemctl enable --now nftables
```

> Appliquer **d'abord sur un seul nœud**, vérifier `check-galera.sh` et l'accès SSH, puis généraliser. Une erreur bloque SSH : garder la console Proxmox à portée.

HAProxy

Mesure	Statut
Page de stats protégée par mot de passe **et** ACL vADMIN	En place
`stats hide-version`	En place
`chroot /var/lib/haproxy`, compte `haproxy` non privilégié	En place (défaut Debian)
En-tête `X-Served-By`	En place pour la démonstration — **à retirer en production** (révèle l'architecture interne)
TLS (HTTPS)	Futur

Apache / PHP

Mesure	Statut
`ServerTokens Prod`, `ServerSignature Off` (`/etc/apache2/conf-available/security.conf`)	Recommandé

Mesure	Statut
Accès direct refusé à `wp-config.php`, `readme.html`	En place (vhost)
`DISALLOW_FILE_EDIT` dans WordPress	En place
`expose_php = Off` dans `php.ini`	Recommandé
GLPI : `DocumentRoot` sur `public/`, `install.php` supprimé	En place

MariaDB

Mesure	Statut
`bind-address` sur l'IP du VLAN vBDD (pas `0.0.0.0`)	En place
`mariadb-secure-installation`	En place
root uniquement via unix_socket	En place (défaut Debian)
Chiffrement TLS de la réplication Galera	Futur

pfSense

Mesure	Statut
Mot de passe admin changé	En place
Interface web accessible depuis vADMIN uniquement	En place
Deny-by-default sur les interfaces internes	En place
Sauvegarde chiffrée de `config.xml`	Recommandé

Sécurité — Moindre privilège

Chaque compte ne dispose que des droits strictement nécessaires à sa fonction.

Comptes SQL

Compte	Hôte	Privilèges	Justification
`root`	`localhost` (unix_socket)	Tous	Administration locale via `sudo mariadb` uniquement ; pas d'accès réseau
`wordpress`	`10.0.10.2`	`ALL ON wordpress.*`	WordPress crée/modifie ses tables lors des mises à jour
`glpi`	`10.0.10.2`	`ALL ON glpi.*` + `SELECT ON mysql.time_zone_name`	Idem pour GLPI + fuseaux horaires
`haproxy_check`	`10.0.10.2`	Aucun (`USAGE`)	Health-check : ouverture de session seulement

Ce qui est **évité** :

- `GRANT ALL ON *.*` à un compte applicatif ;
- l'hôte `'%'` (n'importe quelle origine) ;
- le compte `root` utilisé par une application.

Contrôle

SUR bdd-01

```
SELECT user, host FROM mysql.user ORDER BY user;
SHOW GRANTS FOR 'wordpress'@'10.0.10.2';
-- Aucun compte applicatif avec l'hôte '%' :
SELECT user, host FROM mysql.user WHERE host = '%';
```

Le cas `haproxy_check`

Ce compte sans mot de passe est accessible à quiconque peut joindre `10.0.10.2:3306` (toutes les connexions via HAProxy apparaissent venir de 10.0.10.2). Il ne permet que `SELECT 1`, `SHOW STATUS` et la lecture de quelques variables. Mesures de limitation :

- pfSense : seuls vWEB et vADMIN atteignent `10.0.10.2:3306` ;
- limite de connexions :

`sql ALTER USER 'haproxy_check'@'10.0.10.2' WITH MAX_USER_CONNECTIONS 10;`

- amélioration future : check *wsrep-aware* HTTP (port 9200) qui supprime le besoin de ce compte.

Comptes système

Compte	Usage	Shell	sudo
`<ADMIN_USER>` (par VM)	Administration humaine	`/bin/bash`	Oui
`haproxyadmin` (haproxy)	Administration humaine de HAProxy	`/bin/bash`	Oui
`haproxy`	Service HAProxy (paquet)	`nologin`	Non
`mysql`	Service MariaDB (paquet)	`nologin`	Non
`www-data`	Apache / PHP	`nologin`	Non

Séparer le compte **humain** du compte **de service** évite qu'une faille dans un service donne un shell interactif, et évite le conflit décrit dans INC-08.

Fichiers

Fichier	Propriétaire	Droits
`/var/www/wordpress/wp-config.php`	`root:www-data`	`640`

Fichier	Propriétaire	Droits
`/var/www/glpi/config/config_db.php`	`www-data:www-data`	`640`
`/etc/haproxy/haproxy.cfg`	`root:root`	`644` (contient le mot de passe des stats : `640 root:haproxy` recommandé)
`~/.ssh/id_ed25519`	utilisateur	`600`
`/var/backups/mariadb/*.sql.gz`	`root`	`600`

Réseau

Le principe s'applique aussi aux flux : seul **HAProxy** atteint les bases ; seul **vADMIN** atteint SSH ; les postes **vUSER** n'atteignent que les services publiés. Voir Matrice-de-flux.md.

Sécurité — Gestion des secrets

Inventaire des secrets de l'infrastructure

Secret	Où il est utilisé	Où il est stocké	Dans le dépôt
Mot de passe admin pfSense	Interface web pfSense	Gestionnaire de mots de passe	Jamais
Mot de passe `wordpress` @ `10.0.10.2`	`wp-config.php` (web-01, web-02)	Gestionnaire + fichier `640` sur les VM	`<SECRET>`
Clés/sels WordPress	`wp-config.php`	Fichier sur les VM	`<SECRET>`
Mot de passe `glpi` @ `10.0.10.2`	`config_db.php` (glpi)	Gestionnaire + fichier sur la VM	`<PASSWORD>`
Mot de passe des stats HAProxy	`haproxy.cfg`	Gestionnaire + fichier sur la VM	`<PASSWORD>`
Clé privée SSH d'administration	adm-01, poste admin	`~/.ssh/` (`600`), phrase de passe	Jamais
Comptes WordPress / GLPI	Interfaces web	Gestionnaire de mots de passe	Jamais
Export `config.xml` pfSense	Restauration	Stockage chiffré hors lab	Jamais

Le compte `haproxy_check` n'a **pas** de mot de passe par conception (aucun privilège, restreint à `10.0.10.2`) : ce n'est pas un secret, mais une surface à limiter (voir least-privilege.md).

Règles

1. **Aucun secret dans Git**, même dans un dépôt privé, même « temporairement » : l'historique est conservé.
2. Configurations publiées en `.example` avec placeholders.
3. Mots de passe **uniques** par service, générés aléatoirement (≥ 20 caractères).
4. Mots de passe **demandés à l'exécution** par les scripts (`read -s`), jamais en argument (`-pMotDePasse` est visible dans `ps` et l'historique).

5. Fichiers contenant des secrets sur les VM : propriétaire `root` ou compte de service, droits `600` / `640`.

Garde-fous du dépôt

Outil	Rôle	Coût
`.gitignore`	Empêche l'ajout accidentel de clés, `.env`, sauvegardes, configs réelles	Aucun
`check-repository.sh` (dépôt GitHub)	Recherche fichiers sensibles et motifs de secrets (local + CI)	Aucun
GitHub Secret Scanning	Détecte les formats de tokens connus (GitHub, AWS, Slack…) dans tout l'historique	Gratuit pour les dépôts publics
Push Protection	**Bloque** un push contenant un token reconnu	Gratuit pour les dépôts publics
Dependabot	Tient à jour les versions des GitHub Actions (dependabot.yml)	Aucun

Activer Secret Scanning et Push Protection

Settings → Code security → Secret Protection (intitulé variable selon l'interface GitHub) → activer **Secret scanning** et **Push protection**.

Et CodeQL ?

CodeQL analyse du code applicatif (JavaScript, Python…) à la recherche de vulnérabilités. Ici, seul le générateur contient du JavaScript, exécuté localement sans serveur ni entrée distante. CodeQL n'apporterait qu'un gain marginal pour une complexité de CI supplémentaire : **non retenu** en v1.0.0. Il peut être activé en un clic (*Code scanning → Default setup*) si le générateur évolue.

Limites

Ces outils reconnaissent des **formats** (préfixes de tokens, en-têtes de clés). Un mot de passe comme `Lab2026!` dans un fichier de configuration ne sera détecté ni par GitHub ni par la plupart des scanners. La vraie protection reste la **discipline** : `.example`, placeholders, relecture du `git diff` avant commit.

Avant chaque commit

```
# SUR LE POSTE DE DÉVELOPPEMENT
git diff --cached
make check
```

En cas de fuite

Procédure : `SECURITY.md` (dépôt GitHub) — **changer le secret d'abord**, nettoyer l'historique ensuite.